**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

Лабораторная работа
“ЛР1: Реализация REST API на основе boilerplate”

Выполнил:

Преображенский Артемий

Группа:

K3341

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

**Задача**

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и мой)
2. С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:
3. модели
4. представления
5. контроллеры
6. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

**Ход работы**

В рамках выполнения лабораторной работы была продолжена разработка backend-приложения для бронирования столиков в ресторанах. За основу была взята собственная структура проекта на Go, реализованная в подходе vertical slice architecture.

На верхнем уровне проекта были выделены основные директории:
cmd — точка входа в приложение;
config — конфигурация приложения;
deploy — Docker-конфигурация;
docs — документация и OpenAPI-схема;
internal — внутренняя логика приложения;
migrations — SQL-миграции базы данных;
pkg — переиспользуемые пакеты.

Внутри директории internal были реализованы:
adapter — подключение к PostgreSQL;
controller/http — маршрутизация HTTP-запросов;
domain — доменные модели приложения;
features — отдельные функциональные срезы приложения.

В директории domain были описаны основные модели предметной области: пользователь, ресторан, бронирование, столик, блюдо, отзыв, а также ошибки домена. Эти модели были реализованы на основе ранее спроектированной ER-диаграммы из ДЗ1.

Основная бизнес-логика была распределена по функциональным модулям в директории features. Были реализованы следующие группы возможностей:

1. auth — регистрация, вход и получение профиля пользователя;
2. restaurant — получение списка ресторанов, просмотр конкретного ресторана, удаление ресторана;
3. menu — получение списка блюд ресторана;
4. review — создание, получение, обновление, удаление и просмотр списка отзывов;
5. table — получение списка столиков ресторана;
6. booking — проверка доступности столика, создание бронирования, просмотр списка бронирований, получение конкретного бронирования и отмена бронирования.

booking.go — restaurant-booking

booking.go

```go
package domain

import (
    "time"

    "github.com/google/uuid"
)

type BookingStatus string

type Booking struct {
    ID           uuid.UUID     `json:"id"`
    UserID       uuid.UUID     `json:"user_id"`
    RestaurantID uuid.UUID     `json:"restaurant_id"`
    TableID      uuid.UUID     `json:"table_id"`
    GuestsCount  int           `json:"guests_count"`
    BookingDate  string        `json:"booking_date"`
    StartTime    string        `json:"start_time"`
    EndTime      string        `json:"end_time"`
    Status       BookingStatus `json:"status"`
    CreatedAt    time.Time     `json:"created_at"`
    UpdatedAt    time.Time     `json:"updated_at"`
}
```

Каждый функциональный срез включал файлы dto.go, http.go, usecase.go и postgres.go.

1. В dto.go были описаны структуры запросов и ответов.

2. В http.go была реализована обработка HTTP-запросов и работа с кодами ответов.
3. В usecase.go размещалась бизнес-логика конкретного сценария.
4. В postgres.go была реализована работа с базой данных для соответствующей функции.

postgres.go — restaurant-booking

booking.go | postgres.go

```go
package login

import (
    "context"
    "errors"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"

    "restaurant-booking/internal/adapter/postgres"
    "restaurant-booking/internal/domain"
)

type postgresRepository struct {
    pool *postgres.Pool
}

func NewPostgres(pool *postgres.Pool) *postgresRepository {
    return &postgresRepository{pool: pool}
}

func (r *postgresRepository) FindForLogin(ctx context.Context, email domain.Email) (domain.User, error) {
    var u domain.User
    var phone *string
    err := r.pool.Pgx().QueryRow(ctx, `
        SELECT id, email, password_hash, full_name, phone, created_at, updated_at
        FROM users WHERE email = $1
    `, string(email)).Scan(
        &u.ID,
        &u.Email,
        &u.Password,
        &u.Name,
        &phone,
        &u.CreatedAt,
        &u.UpdatedAt,
    )
    if err != nil {
```

usecase.go — restaurant-booking

booking.go | postgres.go | usecase.go

```go
type Repository interface {
}

type Usecase struct {
    repo   Repository
    jwtCfg jwt.Config
}

func NewUsecase(repo Repository, jwtCfg jwt.Config) *Usecase {
    return &Usecase{repo: repo, jwtCfg: jwtCfg}
}

func (u *Usecase) Login(ctx context.Context, input Input) (Output, error) {
    email := strings.TrimSpace(input.Email)
    if email == "" || input.Password == "" {
        return Output{}, domain.ErrInvalidInput
    }
    user, err := u.repo.FindForLogin(ctx, domain.Email(email))
    if err != nil {
        if errors.Is(err, domain.ErrNotFound) {
            return Output{}, domain.ErrUnauthorized
        }
        return Output{}, err
    }
    if err := bcrypt.CompareHashAndPassword([]byte(user.Password), []byte(input.Password)); err != nil {
        return Output{}, domain.ErrUnauthorized
    }
    profile := user
    profile.Password = ""
    token, err := jwt.Sign(u.jwtCfg, user.ID)
    if err != nil {
        return Output{}, err
    }
    return Output{Token: token, User: profile}, nil
}
```

Также были настроены миграции базы данных, соответствующие ранее спроектированной схеме. Это позволило подготовить структуру таблиц и связей между сущностями для работы API.

000001_init.up.sql — restaurant-booking

booking.go | postgres.go | usecase.go | 000001_init.down.sql | 000001_init.up.sql

Run on active connection | Select block

```sql
CREATE EXTENSION IF NOT EXISTS pgcrypto;

DO $$
BEGIN
    CREATE TYPE cuisine_type AS ENUM (
        'italian',
        'japanese',
        'georgian',
        'russian',
        'american',
        'mexican',
        'chinese',
        'thai',
        'indian',
        'european',
        'mediterranean',
        'other'
    );
EXCEPTION
    WHEN duplicate_object THEN NULL;
END
$$;

DO $$
BEGIN
    CREATE TYPE price_category AS ENUM (
        'low',
        'medium',
        'high'
    );
EXCEPTION
    WHEN duplicate_object THEN NULL;
END
$$;

DO $$
```

Для авторизации был выделен отдельный пакет pkg/jwt, для middleware — отдельная директория pkg/middleware, для логирования — pkg/logger, а для формирования JSON-ответов и ошибок — pkg/render.

token.go — restaurant-booking

booking.go | postgres.go | usecase.go | 000001_init.down.sql | 000001_init.up.sql | token.go

```go
package jwt

import (
    "time"

    jwtlib "github.com/golang-jwt/jwt/v5"
    "github.com/google/uuid"
)

type Config struct {
    Secret  []byte
    Expires time.Duration
}

func Sign(cfg Config, userID uuid.UUID) (string, error) {
    now := time.Now()
    claims := jwtlib.RegisteredClaims{
        Subject:   userID.String(),
        ExpiresAt: jwtlib.NewNumericDate(now.Add(cfg.Expires)),
        IssuedAt:  jwtlib.NewNumericDate(now),
    }
    t := jwtlib.NewWithClaims(jwtlib.SigningMethodHS256, claims)
    return t.SignedString(cfg.Secret)
}

func ParseUserID(cfg Config, tokenString string) (uuid.UUID, error) {
    t, err := jwtlib.ParseWithClaims(tokenString, &jwtlib.RegisteredClaims{}, func(t *jwtlib.Token) (interface{}, error) {
        return cfg.Secret, nil
    })
    if err != nil {
        return uuid.Nil, err
    }
    claims, ok := t.Claims.(*jwtlib.RegisteredClaims)
    if !ok || !t.Valid {
        return uuid.Nil, jwtlib.ErrTokenInvalidClaims
    }
    if claims.Subject == "" {
```

## Вывод

В ходе выполнения лабораторной работы было реализовано backend-приложение на Go. Были разработаны модели, представления и контроллеры, а также реализованы основные бизнес-сценарии приложения согласно материалам ДЗ1 и ДЗ2. Полученное API является работоспособной основой для дальнейшего развития и тестирования проекта.